

Send a message

GET STARTED

Welcome to SP-API Documentation

What is the Selling Partner API?

Selling Partner API Onboarding Overview >

Terminology

Frequently Asked Questions >

CHANGELOG

SP-API Release Notes

SP-API Deprecation Schedule

SP-API Product Metadata Updates

DIRECTORIES

SP-API Models

SP-API Seller Use Cases

Send a message

Learn how to use the Messaging API to send a message to a buyer.

Learn how to use the Messaging API to make an application that lets a seller send a message to a buyer.

Prerequisites

To complete this tutorial, you need:

- Authorization from the selling partner for whom you are making calls. For more information, refer to [Authorizing Selling Partner API applications](#).
- The [Buyer Communication role](#) assigned to your developer profile.
- The [Buyer Communication role](#) role selected in the app registration page for your application.

Step 1. Get the order for which the seller wants to send a message

1. Display a list of the seller's recent orders that the seller can choose from.
2. The seller chooses an order for which they want to send a message to the buyer.
3. Associate an Amazon order identifier with the order that the seller chooses.
4. Save the Amazon order identifier for [Step 2. Get a list of available message types](#).

Step 2. Get a list of available message types

Call the `getMessagingActionsForOrder` operation of the Messaging API, specifying the Amazon order identifier from [Step 1. Get the order for which the seller wants to send a message](#) to get a list of message types that are available for the order.

Path Parameter:

Parameter	Description	Required
amazonOrderId	An Amazon order identifier. This specifies the order for which a message is to be sent. Get this value from Step 1. Get the order for which the seller wants to send a message . Type: string	Yes

Query Parameter:

Parameter	Description	Required
-----------	-------------	----------

SP-API Vendor Use Cases

Seller Central URLs

Vendor Central URLs

REGISTRATION

SP-API Registration Overview >

Developer Registration Request Status

Third-Party Provider Registration >

Service Provider Registration and Role Approval Process Overview

Website Guidelines for Public Developers and Service Providers

User Permissions for Service Providers

Register your Application

Update your Application Information

Rotate your Application's LWA Credentials >

View your Application Information and Credentials

Learn How Sellers Authorize Service Providers

Selling Partner API Roles >

Policies and Agreements

About Facial Data

AUTHORIZATION

Authorize Applications >

Renew Authorizations

Revoke Authorizations

Authorization Limits

Special Authorization Types >

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs >

marketplace
Ids

A marketplace identifier. This specifies the marketplace in which the order was placed. You can specify only one marketplace.

Type: array[string]

Yes

2. Save the list of schemas in the `_embedded.actions` array of the response. In the `_embedded.schema` section of each array element you can find a schema for each available message type. Use the `title` value in each schema to create a list of available message types for the seller to choose from in [Step 3. Get the seller's choice of message type](#). Use the schemas to understand how to display and enforce the restrictions for custom messages and attachments that apply for each message type. For more information, refer to [Step 4. Get custom message text from the seller \(if supported\)](#) and [Step 5. Get an attachment from the seller \(if supported\)](#).

3. Save the list of paths in the `_links.actions` array of the response. The `href` value in each array element contains a path for each available message type. You can use one of these paths in a call for the appropriate operation in [Step 9. Send a message to the buyer](#).

Step 3. Get the seller's choice of message type

1. Display a list of available message types for the seller to choose from. Get these from the `title` values that you saved in [Step 2. Get a list of available message types](#).
2. Let the seller choose the message type that they want send, and save it. This choice will affect several subsequent steps, including:
 1. [Step 4. Get custom message text from the seller \(if supported\)](#). The choice of message type determines whether a custom message is supported.
 2. [Step 5. Get an attachment from the seller \(if supported\)](#). The choice of message type determines whether attachments are supported.
 3. [Step 9. Send a message to the buyer](#). The choice of message type determines which operation you will call in Step 9. For example if the seller chooses the "Confirm customization details" message type, you will call the `confirmCustomizationDetails` operation in Step 9.

Step 4. Get custom message text from the seller (if supported)

Some message types permit the seller to include a custom message in the email to the buyer. To find out if a message type supports custom messages, review the Messaging API reference to find the operation that corresponds to the message type. For example, the `createConfirmServiceDetails` operation corresponds to the "Contact Home Service customer" message type. If the operation includes a text body parameter, the message type supports custom messages.

1. Determine if the message type that the seller chose in [Step 3. Get the seller's choice of message type](#) supports custom messages. If custom messages are not supported, go directly to [Step 5. Get an attachment from the seller \(if supported\)](#). If custom messages are supported, continue with this procedure.
2. Let the seller input the raw text that they want for their custom message.
3. Display and enforce any restrictions on the message text that the seller inputs. You can find the text restrictions in the `_embedded.schema.properties.rawMessageBody` section of the `_embedded.actions` array element that corresponds to the seller's message type. Find this in the response for the `getMessagingActionsForOrder` operation in [Step 2. Get a list of available message types](#).
4. Save the message text and use it as input for [Step 9. Send a message to the buyer](#).

Step 5. Get an attachment from the seller (if supported)

Some message types permit the seller to include a link to an attachment in the email to the buyer. To find out if a message type supports attachments, review the Messaging API reference to find the operation that corresponds to the message type. For example, the `createWarranty` operation corresponds to the "Send Warranty Information" message type. If the operation includes an attachments body parameter, the message type supports attachments.

1. Determine if the message type that the seller chose in [Step 3. Get the seller's choice of message type](#) supports attachments. If attachments are not supported, go directly to [Step 9. Send a message to the buyer](#). If attachments are supported, continue with this procedure.
2. Let the seller upload an attachment to your application.
3. Display and enforce any restrictions on the attachment that the seller uploads. You can find the attachment restrictions in the `_embedded.schema.properties.attachments` section of the `_embedded.actions` array element that corresponds to the seller's message type. Find this in the response for the `getMessagingActionsForOrder` operation in [Step 2. Get a list of available message types](#).
4. Convert the attachment into an input stream and save it for [Step 8. Upload the attachment](#).
5. Let the seller input a file name for the attachment, including the file extension.
6. Save the file name and use it as input for [Step 9. Send a message to the buyer](#).

Step 6. Calculate a Content-MD5 hash

Calculate a Content-MD5 hash for the attachment from [Step 5. Get an attachment from the seller\(if supported\)](#). You will need it to create an upload destination in [Step 7. Create an upload destination](#).

The Java sample code in this step contains logic for calculating a Content-MD5 hash. This sample code uses the Apache HTTP client.

1. Use the following as input for the sample code:
2. The attachment input stream from [Step 5. Get an attachment from the seller \(if supported\)](#), is the argument for the `fis` parameter of the `computeContentMD5Value` method of the `CreateMD5` class.
3. Save the `md5Content` value to pass in with the `Content-MD5` parameter in [Step 7. Create an upload destination](#).

Sample Java code

```
Java
package io.swagger.client.util;

import java.io.FileInputStream;
import java.io.IOException;
import java.security.DigestInputStream;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class CreateMD5
{
    public static String computeContentMD5Value(FileInputStream fis)
        throws IOException,
        NoSuchAlgorithmException
    {
        DigestInputStream dis = new DigestInputStream(fis,
            MessageDigest.getInstance("MD5"));
    }
}
```

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

SOLUTION PROVIDER PORTAL

- Manage Users in Solution Provider Portal
- Manage Your Business And Contact information in Solution Provider Portal
- Update Your Developer Profile in Solution Provider Portal
- Unify Your Developer Accounts
- Verify Your Identity in Solution Provider Portal
- API Usage Metrics in Solution Provider Portal

TUTORIALS

- Tutorial: Subscribe to the ORDER_CHANGE Notification
- Tutorial: Test Selling Partner API Endpoints
- Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Tutorial: Automate your SP-API Calls Using a Python SDK

Tutorial: Automate your SP-API Calls using a prebuilt C# SDK

Tutorial: Automate your SP-API Calls using prebuilt Java SDK

Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK

Tutorial: Automate your SP-API Calls using prebuilt PHP SDK

Tutorial: Automate your SP-API calls using a prebuilt Python SDK

Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application

Tutorial: Retrieve Merchant Shipping Templates

Tutorial: Grant the SP-API Permission to an Amazon SQS Queue

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors [↗](#)

Listings Items API Issues Troubleshooting [↗](#)

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

Edit Your Appstore Listing

Check Listing Status

Appstore Ratings and Reviews

Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview

Amazon Selling Partner API Guard Implementation Guide [➤](#)

```
byte[]buffer = new byte[8192];
while (dis.read(buffer) > 0);

String md5Content = new String(org.apache.commons.codec.binary.Base64
    .encodeBase64(dis.getMessageDigest().digest()));

// Effectively resets the stream to be beginning of the file
// via a FileChannel.
fis.getChannel().position(0);

return md5Content;
}
}
```

Step 7. Create an upload destination

Call the `createUploadDestinationForResource` operation of the Uploads API to create an upload destination for your attachment, in the form of an Amazon S3 presigned URL.

Query parameters:

Type	Name	Description	Schema
Query	marketplaceIds <i>required</i>	A list of marketplace identifiers. This specifies the marketplaces where the upload will be available. Only one marketplace can be specified. Max count : 1	< string > array
Query	contentMD5 <i>required</i>	The Content-MD5 hash that you calculated in Step 6. Calculate a Content MD5 hash.	string
Path	resource <i>required</i>	The resource for the upload destination that you are creating. For example, if you are creating an upload destination for the <code>createLegalDisclosure</code> operation of the Messaging API, the <code>{resource}</code> would be <code>/messages/v1/orders/{amazonOrderId}/messages/legalDisclosure</code> , and the entire path would be <code>/uploads/2020-11-01/uploadDestinations/messaging/v1/orders/{amazonOrderId}/messages/legalDisclosure</code> . If you are creating an upload destination for an Aplus content document, the <code>{resource}</code> would be <code>aplus/2020-11-01/contentDocuments</code> and the path would be <code>/uploads/v1/uploadDestinations/aplus/2020-11-01/contentDocuments</code> .	string
Query	contentType <i>optional</i>	The content type of the file to be uploaded.	string

Save the following values:

- **url** and **headers**. Use these values in [Step 8. Upload the attachment.](#)
- **uploadDestinationId**. Use this value in [Step 9. Send a message to the buyer.](#)

Step 8. Upload the attachment

Upload to an Amazon S3 bucket the attachment that the seller uploaded to your application in [Step 5. Get an attachment from the seller \(if supported\).](#)

The Java sample code in this step contains logic for uploading an attachment. This sample code uses the Apache HTTP client.

Use the following as input to the sample code:

- Use the **url** value that you saved in [Step 7. Create an upload destination](#) as the argument for `URL(uploadDestinationPayload.getPayload().getUrl())` in the `UploadFileToDestination` class.
- Use the **headers** value that you saved in [Step 7. Create an upload destination](#) as input for `(Map < String, String >)uploadDestinationPayload.getPayload().getHeaders()` in the `UploadFileToDestination` class.
- Use the attachment input stream that you saved in [Step 4. Get custom message text from the seller \(if supported\)](#) and use it as input for the `out.write()` method of the `UploadFileToDestination` class.

Sample Java code

```
Java
package io.swagger.client.util;

import java.io.IOException;
import java.io.OutputStreamWriter;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Map;

import io.swagger.client.model.CreateUploadDestinationResponse;

public class UploadFileToDestination
{
    private static String requestMethod = "PUT";
    private static String contentType = "Content-Type";
    private static String contentValue = "application/text";
    private static URL url;
    private static Map < String,
String > headers;

    /* Upload file to destination using the response returned in
CreateUploadDestination. Reference
https://docs.aws.amazon.com/AmazonS3/latest/dev/PresignedUrlUploadObjectJavaSDK
*/

    public static void uploadFileInDestination(
        CreateUploadDestinationResponse uploadDestinationPayload)
        throws IOException
    {
        url = new URL(uploadDestinationPayload.getPayload().getUrl());
        headers = (Map < String, String >
)uploadDestinationPayload.getPayload().
        .getHeaders();

        // Create the connection and use it to upload the new object using the
// pre-signed URL.
        HttpURLConnection connection = (HttpURLConnection)url.openConnection();
        connection.setDoOutput(true);
        connection.setRequestMethod(requestMethod);
        connection.setRequestProperty(contentType, contentValue);
        // headers that are returned in createDestination call
        for (Map.Entry < String, String > headersEntry: headers.entrySet())
        {
            connection.setRequestProperty(headersEntry.getKey(),
headersEntry.getValue());
        }
        OutputStreamWriter out = new OutputStreamWriter(
            connection.getOutputStream());
```

VAT Calculation Service



Amazon Seller Data Access

Best Practices



A+ CONTENT API

A+ Content API



A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API

Amazon Warehousing and Distribution API



Amazon Warehousing and Distribution API v2024-05-09
Reference

APP INTEGRATIONS API

App Integrations API



APPLICATION MANAGEMENT API	
Application Management API	>
CATALOG ITEMS API	
Catalog Items API	>
Catalog Items API Rate Limits	
Catalog Items API v2022-04-01 Reference	
CUSTOMER FEEDBACK API	
Customer Feedback API	>
Customer Feedback API Rate Limits	
DATA KIOSK	
Data Kiosk API	>
Data Kiosk Schema Explorer User Guide	
Data Kiosk Query Processing Finished Notification	
Building Data Kiosk workflows guide	
Vendor Analytics Dataset Use Case Guide	
Schedule Data Kiosk Queries	
DELIVERY BY AMAZON API	
Delivery by Amazon API	>
EASY SHIP API	
Easy Ship API	>
EXTERNAL FULFILLMENT	
External Fulfillment Inventory API	>

Send a message

```
out.write("Text of the file");
out.close();
// Check the HTTP response code. To complete the upload and make the
// object available,
// you must interact with the connection object in some way.
connection.getResponseCode();
System.out.println("HTTP response code: " +
connection.getResponseCode()
+ connection.getResponseMessage());
connection.disconnect();
}
```

The attachment is uploaded to the Amazon S3 bucket.

Step 9. Send a message to the buyer

To send a message from the seller to the buyer you need to call the operation that corresponds to the message type that the seller chose in [Step 2. Get a list of available message types](#). For example, if the seller chose the "Confirm customization details" message type, you would call the [confirmCustomizationDetails](#) operation. Refer to the Messaging API reference for the list of operations that are available. In this example, we will call the [confirmCustomizationDetails](#) operation.

Path parameter:

Parameter	Description	Required
amazonOrderId	An Amazon order identifier. This specifies the order for which a message is to be sent. Use the Amazon order identifier that you saved in Step 1. Get the order for which the seller wants to send a message . Type: string	Yes

Query Parameter:

Parameter	Description	Required
marketplaceIds	A marketplace identifier. This specifies the marketplace in which the order was placed. You can specify only one marketplace. Type: array[string]	Yes

Body Parameter:

Parameter	Description	Required
text	The custom message to be included in the email to the buyer. Use the message text from Step 4. Get custom message text from the seller (if supported) . Type: string	Yes
attachments	Attachments to include in the message to the buyer <code>uploadDestinationId</code> . The identifier of the upload destination. Use the <code>uploadDestinationId</code> value from Step 7. Create an upload destination. <code>fileName</code> . The name of the file, including the extension. This file name appears in the email message. Use the <code>fileName</code> value from Step 5. Get an attachment from the seller (if supported). Type: object	Yes

There is no response. Instead, Amazon emails the seller's message to the buyer.

External Fulfillment Returns API >

Updated 19 days ago

External Fulfillment Shipping API >

← Messaging API

Notifications API →

External Fulfillment Errors

External Fulfillment APIs Rate Limits

Did this page help you? ☐ Yes ☐ No

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API >

[Policies and Agreements](#) [Privacy notice](#) | [Conditions of use](#)

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Feeds API >

Feed Type Values >

Feeds JSON Schemas ↗

Feeds API Best Practices

Feeds API FAQ

Feeds API Rate Limits

FINANCES

Finances API >

Transfers API >

FULFILLMENT INBOUND API

Fulfillment Inbound API >

Fulfillment Inbound API FAQ

Migrating Fulfillment Inbound workflows

Fulfillment Inbound API Rate Limits

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide

Fulfillment Outbound API >

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API >

Invoices API FAQ

LISTINGS

Listings Items API >

Listings Items API Rate Limits

Listings Restrictions API >

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration >

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API >

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

MESSAGING API

Messaging API >

[Send a message](#)

NOTIFICATIONS API

Notifications API >

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification ↗

ORDERS API

Orders API >

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier ↗

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API >

PRODUCT PRICING API

Product Pricing API >

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide

Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API >

REPORTS API

Reports API >

Report Type Values >

Reports JSON Schemas ↗

Reports API FAQ

SALES API

Sales API >

SELLER WALLET API

Seller Wallet API >

Seller Wallet API Rate Limits

SELLERS API

Sellers API >

SERVICES API

Services API >

SHIPMENT INVOICING API

Shipment Invoicing API >

SHIPPING API

Shipping API v2 Resources ↗

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API >

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API >

